

Neural Networks

The building block of deep learning: many simple units that together learn complex patterns.

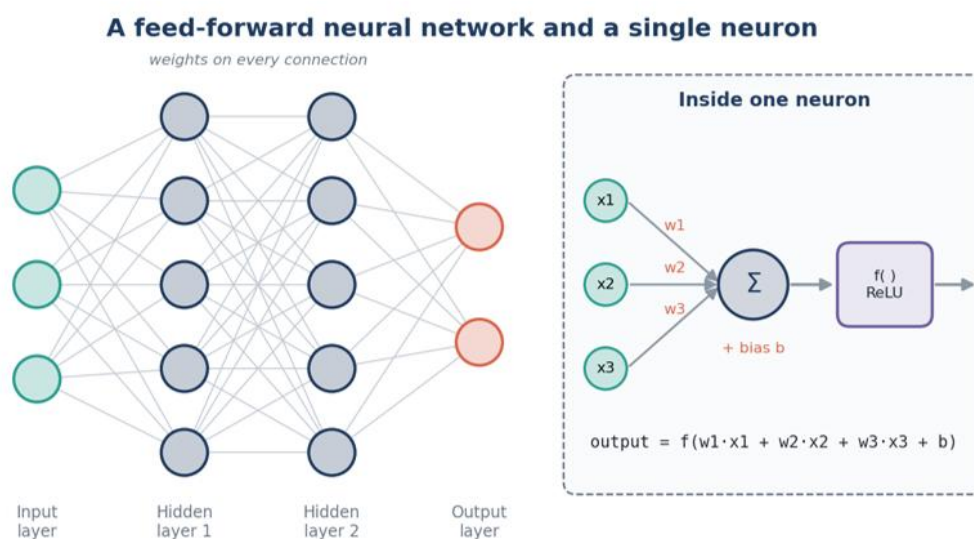
Explanation

A **neural network** is a stack of simple calculators called **neurons**. Each neuron takes some numbers in, multiplies each by an importance score (a **weight**), adds them up with a small adjustment (a **bias**), and passes the result through an **activation function** that decides how strongly it "fires".

5 key terms

Term	What it means
1. Neuron	The basic unit of a network: it computes a weighted sum of its inputs plus a bias, then applies an activation function.
2. Weight	A learnable number on each connection that scales how much an input influences a neuron. Weights are the "knowledge" of the model.
3. Bias	A number that helps the neuron adjust its answer.
4. Activation function	A rule that helps the neuron decide what output to give. A non-linear function that lets networks model complex, non-linear relationships such as ReLU(Rectified Linear Unit).
5. Backpropagation	An algorithm that helps the network learn from its mistakes by changing the weights.

Architecture diagram



How it learns

A brand-new network knows nothing. All its weights start as random numbers, so its first answers are pure guesses. It learns from examples where we already know the right answer, such as thousands of photos already labelled "cat" or "dog". Training repeats one simple loop:

1. **Guess.** Show the network an example. The numbers flow forward through the layers and it gives an answer, for example “70% sure this is a dog”.
2. **Check.** Compare the guess with the right answer. A score called the **loss** measures how wrong it was: confidently wrong gives a big loss, nearly right gives a small one.
3. **Find the blame.** **Backpropagation** works backwards through the network to figure out how much each weight contributed to the mistake.
4. **Adjust.** Every weight is nudged a tiny step in the direction that would have made the answer less wrong. This step is called **gradient descent**.
5. **Repeat.** Do this for every example, many times over. Each full pass through all the examples is called an **epoch**.

An everyday comparison: think of adjusting the hot and cold taps in a shower. You start at a random setting, feel the water, turn the taps slightly in the right direction, and repeat until it feels right. The weights are the taps, the loss is how far the water is from the temperature you want, and nobody tells the network the right settings. It finds them by trial and correction.

Two things decide whether training works:

- **Enough good examples.** The network can only learn patterns that appear in its training data, so more varied, accurate examples mean better results.
- **Testing on unseen examples.** We always hold back some examples the network never trains on. If it does well on those too, it has genuinely learned the pattern. If it only does well on the training examples, it has just memorised them. This is called **overfitting**, like a student who memorises past exam answers but can’t handle a new question.

Where it is used

- Computer vision: photo tagging, OCR, medical scans, quality inspection on production lines.
- Speech: voice assistants, transcription, speaker identification.
- Language: translation, search ranking, and every modern LLM.
- Tabular and time series: forecasting, recommendation systems, fraud scoring at scale.
- Check out this real world example which trains a neural network model [Handwriting Recognition](#) .